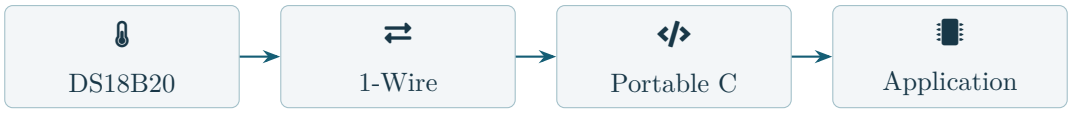
Embedded Drivers

Learn, build, and reuse

A practical workbook and project report

From the first temperature reading to a library you can use again.



The learning path

Connect the board. Read a temperature. Understand each function. Move the library into your own project.

First project: DS18B20 on STM32F107VCT6 / PB10

Keil · PlatformIO · CubeMX

Author: **Alireza Khajehvandi**

Revision: 2026-10-12

Alireza Khajehvandi – Online profiles & contact

Portfolio · GitHub · LinkedIn

YouTube · Telegram · Instagram

Contents

1 A collection that grows one project at a time **3**

1.1 Current catalog . . . 3

1.2 Where each part belongs . . . 3

1.3 How to read and reproduce the work . . . 4

2 DS18B20: read your first temperature **5**

2.1 What we will build . . . 5

2.2 Step 1: check three wires and one resistor . . . 5

2.3 Step 2: open, build and download in Keil . . . 6

2.4 Step 3: make Watch show the running program . . . 7

2.5 Step 4: decide whether the display or the sensor has failed . . . 8

2.6 Step 5: use two functions in your own STM32 project . . . 9

2.7 Step 6: understand the complete runnable example . . . 10

2.8 Step 7: change a pin or rebuild from a copied project . . . 12

2.9 Step 8: follow the sensor transaction without reading all the code . . . 13

2.10 Step 9: read the portable function, one block at a time . . . 13

2.11 Step 10: understand the adapter when you want to port it . . . 15

2.12 Step 11: record what the board actually proved . . . 16

3 Add the next independent driver **17**

3.1 Start from a copied foundation . . . 17

3.2 A repeatable development order . . . 17

3.3 Add a chapter to this report . . . 18

About the author and contact **20**

1 A collection that grows one project at a time

I keep this repository for reusable embedded drivers and for explaining how I build them. Each device has a complete project that can be copied to another directory. The portable library can also be taken alone and connected to another MCU through a small board adapter.

The report is shared documentation, not a shared firmware dependency. There is one English edition and one Persian edition. A new driver becomes a new chapter, with its own hardware, code, acceptance procedure and validation record. The project README remains sufficient for using a project without this report.

1.1 Current catalog

Project	Purpose	Current evidence
DS18B20	One-sensor Celsius reading on STM32F107VCT6, PB10	Builds and portable tests; board acceptance pending
STM32F107 foundation	Independent starting project to copy	Consult the template's local status

1.2 Where each part belongs

```
ds18b20/
  DS18B20.ioc          # hardware configuration
  Core/                # CubeMX-generated initialization
  Drivers/             # local ST HAL and CMSIS
  Application/         # board adapter and hardware demo
  Libraries/OneWire/   # portable wire protocol
  Libraries/DS18B20/   # portable temperature operation
  MDK-ARM/             # Keil project and startup
  platformio.ini       # GCC build and probe selection
  tests/               # simulated portable C tests
  tools/               # source synchronization/checks
  README.md            # standalone usage instructions
  PROJECT_STATUS.md    # evidence and unfinished work
docs/
  english/             # one complete report edition
  persian/             # matching report edition
  tools/               # report build and PDF verification
```

`Core` initializes the MCU. `Application` knows the GPIO registers and clock. `Libraries` contains the reusable C code for talking to the sensor. This separation lets the same temperature code run in a Cortex-M3 simulation or on a different board.

1.3 How to read and reproduce the work

For your first result, follow DS18B20 steps 1 through 4 in order. Step 5 teaches the two functions you call in another project; steps 6 and 7 explain the example and pin changes. Steps 8 through 10 explain the implementation after the practical result. You can return to those later. Keep a distinction between a successful compiler run, a passing simulated test and an actual measurement on a board. The final step is always the board acceptance record.

Datasheet PDFs are local working references and are excluded from Git. READMEs and the bibliography preserve official source URLs and revisions. Final report PDFs and editable LaTeX sources are tracked; downloaded fonts, rendered review images and build artifacts are excluded.

2 DS18B20: read your first temperature

2.1 What we will build

Our first task is simple: connect one sensor and see its temperature in Keil. Once that works, we will use the same reading function in another project. Only then will we look inside the driver. You do not need to understand function pointers or the 1-Wire protocol to complete the first steps.

This version supports **one externally powered DS18B20 on each bus**. It reads Celsius and checks errors. It does not change the sensor's resolution, search for multiple sensors, write EEPROM or provide parasite power. Keeping those features out makes the first implementation easier to follow.

At the end of this chapter: you can wire and run the example, distinguish a stale Watch display from a sensor error, use the reading function, change the GPIO through CubeMX and explain how the temperature bytes become a number.

The hardware reference is the official DS18B20 datasheet: document 19-7487, Rev. 6, July 2019 [1].

2.2 Step 1: check three wires and one resistor

The existing example uses STM32F107VCT6. Connect the sensor as follows. Check the package drawing in the datasheet before deciding which physical leg is VDD or GND; a bottom-view drawing must not be read as a top view.

Sensor signal	Board connection
VDD	3.3 V supply
GND	The same ground as the microcontroller
DQ	PB10, the data pin
Pull-up resistor	4.7 kΩ between DQ and 3.3 V

DQ is the only data wire. It must be an open-drain output: the MCU can pull it low or let go of it. The resistor brings a released wire back high. The internal GPIO pull-up is not a substitute for the external resistor in this example.

The project's clock is an 8 MHz external crystal multiplied to a 72 MHz CPU clock. Confirm that the assembled board really has this crystal. The SWD connections are PA13/SWDIO, PA14/SWCLK, ground and probe reference voltage. No display or serial terminal is needed for the first test.

Before proceeding: one sensor is connected, VDD is powered independently, the ground is common, DQ goes to PB10 and the pull-up is present.

2.3 Step 2: open, build and download in Keil

1. Open `ds18b20/MDK-ARM/DS18B20.uvprojx`. If you copied the project elsewhere, open the copy's project file.
2. Install the STM32F1 device pack 2.4.1 and ARM Compiler 6. The compiler used for validation here is 6.24.
3. In Options for Target, open Debug and select the physical probe. Use J-Link for a J-Link probe, or select the correct ST-Link driver for an ST-Link probe. Do not choose the simulator to test the real sensor.
4. Check Utilities uses the target's probe for Flash programming. Build the target. Resolve build errors before starting debug.
5. Download the new firmware, then enter the debug session. Whenever source code changes, build and download again; editing a C file does not change the firmware already on the board.
6. Remove or disable breakpoints in the reading path. Run the program freely.

The application calls initialization once and reading repeatedly. The sensor may need about 750 ms to complete a conversion. The initial numeric zero is only the C variable's starting value; it is not a successful reading.

For PlatformIO, open the `ds18b20` folder and use the matching probe environment:

```
pio run -e stm32f107vc_jlink
pio run -e stm32f107vc_jlink -t upload
pio debug -e stm32f107vc_jlink
# For an ST-Link probe, use stm32f107vc_stlink.
```

PlatformIO Core must be installed. The first build obtains the pinned toolchain. The firmware uses the project's local HAL, CMSIS, startup and linker files.

2.4 Step 3: make Watch show the running program

In the debug session, open Watch 1. Double-click an empty expression row and add these names. Use **the global name** `g_temperature_c`, not the local variable `value` inside a function.

Expression	What it tells you
`g_temperature_c`	Last successful Celsius value
`g_temperature_status`	Result of the last completed attempt
`g_temperature_valid`	Whether that attempt succeeded
`g_measurement_busy`	Whether a new read is in progress
`g_successful_samples`	How many reads actually succeeded
`g_sample_cycles`	How many attempts finished, including errors
`g_last_success_ms`	HAL tick at the last success

Enable View → Periodic Window Update. Keil updates Watch when execution stops; live updates require this option [2]. If it is off, a breakpoint makes a new value appear because stopping refreshes the display. This does not prove that the breakpoint caused the measurement.

This exact option was zero in the saved local options file when the reported symptom was investigated. It has been set to one, and the Watch expressions were added. An already open Keil session retains its loaded settings: enable the menu option in that session too. The ignored local `.uvoptx` is not a shared Git configuration. To reproduce the setup, save and close Keil, then run from the project folder:

```
python tools/setup_keil_watch.py
```

The script backs up local settings and keeps existing probe choices, watches and breakpoints. If Keil later saves old settings from an open session, run the script again after closing it.

A simple example of a successful result is temperature 24.5, status 0, valid true and an increasing success count. The exact temperature depends on the board and environment. A constant temperature can be correct in a steady room; the **success count** tells us whether new reads are

completing.

Validity describes the last *completed* attempt. It stays visible while the next conversion runs. Busy describes that conversion separately. If a completed attempt fails, valid becomes false and the previous number remains in memory. Never use that number as a fresh measurement after an error.

Pass this step: the success count increases during free execution and status remains zero. If only the screen is frozen, check live refresh and the global expression before changing the sensor driver.

2.5 Step 4: decide whether the display or the sensor has failed

1. A value appears only at a breakpoint: first check Periodic Window Update. A stopped display and a stopped CPU are different things.

2. The success count increases but the temperature hardly changes: warm the sensor gently. Compare several samples, not one screen refresh.

3. Attempts increase but successes do not: look at the status code. The loop runs, but the sensor transaction fails.

4. Live values still cannot be read: let the program run, then pause during the 250 ms gap between attempts. Compare the counters at two such observations. Some probe/session configurations may not provide the same live memory access.

5. Counters do not advance even at safe pauses: locate the program counter. A halt in `Error_Handler` indicates startup failure. A halt in a fault handler indicates a CPU exception. A persistent `HAL_Delay` needs a working SysTick and enabled interrupts. Do not infer a driver hang from the Watch value alone.

Do not put a breakpoint inside GPIO slots, microsecond delays or busy polling to obtain live temperature. A debugger pause breaks real-time communication. A breakpoint after the completed read can be useful for one inspection, but is not the normal display mechanism.

Status	Meaning and first action
0, OK	Use the temperature.
1, invalid argument	Check the output pointer and callback binding.
2, no device	Check VDD, GND, DQ, resistor and actual pin.
3, timeout	Check external power, clock and delays.
4, CRC error	Check wiring, noise, rise time, timing and debugger pauses.
5, invalid data	Check sensor type, fixed bytes and a line held low.

2.6 Step 5: use two functions in your own STM32 project

The board adapter handles GPIO and timing through two functions:

- `DS18B20_STM32_Init()`: prepare the adapter once after board setup.
- `DS18B20_STM32_ReadTemperature()`: request a new conversion and read its result.

The following example gives the second function the address of the output variable. There are **two outputs**: the return value says whether the call succeeded; the temperature is written through the pointer. The symbol `&temperature` means "the address of this variable." Do not write `temperature = DS18B20_STM32_ReadTemperature(...)`: that would assign a status number, not Celsius.

`float` stores a decimal number. `DS18B20_Status` stores an operation result, and `DS18B20_OK` means success. The `if` block uses the temperature only after that success.

```
#include "ds18b20_stm32.h"

/* After HAL, clock and GPIO initialization: */
DS18B20_STM32_Init();

/* From a foreground loop, with interrupts enabled: */
float temperature;
DS18B20_Status result = DS18B20_STM32_ReadTemperature(&temperature);
if (result == DS18B20_OK) {
    /* temperature can now be used. */
} else {
    /* Report the failure; do not use temperature here. */
}
```

This small example illustrates the calls. The include goes in the include section, initialization after the board setup, and the read in your loop. It is not a whole `main.c` to paste over the generated file.

To add the implementation to another STM32F107 project:

1. Copy both portable folders: `Libraries/OneWire` and `Libraries/DS18B20`.
2. Copy `Application/Inc/ds18b20_stm32.h` and `Application/Src/ds18b20_stm32.c`.
3. Compile the two portable C files and the adapter C file. Add their three Inc folders to the compiler include paths.
4. Give the new project's open-drain GPIO the CubeMX label `ONE_WIRE_1`. The adapter reads the generated pin macros. Configure a matching CPU clock and keep interrupts enabled when reading.
5. Call Init once after board setup. Call ReadTemperature from your application's normal loop and check its status.

The adapter is specific to this Cortex-M3/register design. A different MCU requires its own adapter and vendor/startup/linker files. The two portable libraries do not require any STM32 header.

2.7 Step 6: understand the complete runnable example

Here is the actual application used by the supplied firmware:

```
#include "app.h"
#include "ds18b20_stm32.h"
#include "main.h"

volatile float g_temperature_c;
volatile bool g_temperature_valid;
volatile bool g_measurement_busy;
volatile DS18B20_Status g_temperature_status = DS18B20_NO_DEVICE;
volatile uint32_t g_sample_cycles;
volatile uint32_t g_successful_samples;
volatile uint32_t g_last_success_ms;

void App_Init(void)
{
```

```
    DS18B20_STM32_Init();
}

void App_Process(void)
{
    float value;
    g_measurement_busy = true;
    DS18B20_Status status = DS18B20_STM32_ReadTemperature(&value);
    /* Publish the latest completed attempt. Keep its validity visible while
     * the next conversion is running; busy reports that conversion separately.
    */
    g_temperature_status = status;
    g_temperature_valid = (status == DS18B20_OK);
    if (g_temperature_valid) {
        g_temperature_c = value;
        ++g_successful_samples;
        g_last_success_ms = HAL_GetTick();
    }
    ++g_sample_cycles;
    g_measurement_busy = false;
    HAL_Delay(250U);
}
```

Read it in this order: `App_Init` calls adapter initialization. `App_Process` marks a read busy, asks for temperature, records the returned status and publishes the successful value. It increments completed attempts, clears busy and pauses before the next attempt.

The temporary `value` is local to this call. The `g_...` variables are global, so Watch can find their fixed RAM locations. They are volatile to ensure the compiler performs the writes. Volatile does not turn on the IDE's refresh option, and these separate fields are not a synchronized RTOS message.

On success, the successful-read count and last-success tick change. On failure, only the completed-attempt count and status/validity change; the old temperature and last-success time stay intact. The new application test checks this sequence and recovery from an error.

The generated main has only three connections to the application:

```
/* USER CODE BEGIN Includes */
#include "app.h"
/* USER CODE END Includes */

/* USER CODE BEGIN 2 */
App_Init();
```

```
/* USER CODE END 2 */

/* USER CODE BEGIN 3 */
App_Process();
/* USER CODE END 3 */
```

The first is in the include area, the second follows HAL/clock/GPIO setup, and the third is inside the generated infinite loop. Keep the generated braces. Use either the supplied `App_Process` or your own reading loop; do not run both readers on the same bus at once.

2.8 Step 7: change a pin or rebuild from a copied project

For a complete independent example, copy the whole `ds18b20` folder. For a new driver project, copy the raw foundation rather than building against it. Every project owns its local IOC, vendor files and build configuration.

For a GPIO change in this example:

1. Open `DS18B20.ioc`. Move the GPIO but preserve `ONE_WIRE_1`, open-drain output, initial high/released level, no internal pull and high speed.
2. Preserve correct clock/SWD settings. In Project Manager, keep user code, copy vendor files locally, use MDK-ARM V5 and the DS18B20 project name. Generate into the same project folder [3].
3. Check that the three application calls above still exist in USER CODE. Application and Libraries are separate from generated Core.
4. Run the following commands, then build Keil and download the new firmware too.

```
python tools/sync_keil.py
python tools/check_project.py
python tests/run_tests.py
pio run -e stm32f107vc_jlink -e stm32f107vc_stlink
```

The post-generation batch hook tries to synchronize/check automatically. Run the commands manually if Python is missing from CubeMX's environment. Synchronization adds the application, adapter and portable sources to Keil; a generated project file alone does not automatically know about every hand-written C file.

Pin-transfer check already performed: a disposable copy moved PB10 to PB11, regenerated and built in both toolchains without editing GPIO literals in the adapter. The delivered example remains PB10.

2.9 Step 8: follow the sensor transaction without reading all the code

Imagine asking the sensor two questions: "Please measure now," then "What was the result?" Both begin with a reset that gets the sensor's attention.

1. Reset and detect presence. Without a response, return no-device.
2. Send `0xCC`: use the only sensor on the wire. Send `0x44`: start a temperature conversion.
3. Poll until conversion completes, waiting between busy polls. Stop waiting when the driver's budget is exhausted.
4. Reset again. Send `0xCC` and `0xBE`: read the scratchpad, the sensor's nine-byte result area.
5. Verify the check byte, decode the temperature and return OK. On a failure, leave the caller's temperature unchanged.

The `0xCC` command is why one sensor is required. With multiple sensors they respond together; this version does not count or individually address them. External VDD allows polling during conversion; parasite power needs a different electrical arrangement.

The temperature uses the first two scratchpad bytes, low byte first. For bytes `91 01`, the value is `0x0191`, or 401. Each raw unit is 1/16 degree, so 401/16 = 25.0625 Celsius. A negative example, `FF5E`, decodes to -162/16 = -10.125 Celsius.

CRC is a check against corrupted bytes. The driver also checks fixed bytes: nine zeros happen to have a valid zero CRC, so CRC alone would accept a stuck-low bus. Resolution controls how many low bits are meaningful; the driver masks undefined bits before converting to Celsius. A valid 85 Celsius is accepted because it can be a real temperature. Repeated implausible 85 needs power/timing investigation.

2.10 Step 9: read the portable function, one block at a time

This is the entire temperature operation. Its C file has no MCU header:

```c
#include "ds18b20.h"
DS18B20_Status DS18B20_ReadTemperature(const DS18B20 *sensor, float *celsius)
{
    uint8_t scratchpad[9];
    uint16_t bits;
    int32_t raw;
    uint32_t elapsed = 0;
    const OneWire_Bus *bus;
    if (!sensor || !celsius || !sensor->delay_ms ||
        !OneWire_IsValid(&sensor->bus)) return DS18B20_INVALID_ARGUMENT;
    bus = &sensor->bus;
    if (!OneWire_Reset(bus)) return DS18B20_NO_DEVICE;
    OneWire_WriteByte(bus, 0xCC); /* Skip ROM: one sensor only. */
    OneWire_WriteByte(bus, 0x44); /* Convert T. */
    while (!OneWire_ReadBit(bus)) {
        if (elapsed >= 800U) return DS18B20_TIMEOUT;
        sensor->delay_ms(bus->context, 1U);
        ++elapsed;
    }
    if (!OneWire_Reset(bus)) return DS18B20_NO_DEVICE;
    OneWire_WriteByte(bus, 0xCC);
    OneWire_WriteByte(bus, 0xBE); /* Read scratchpad. */
    for (uint8_t i = 0; i < 9U; ++i) scratchpad[i] = OneWire_ReadByte(bus);
    if (OneWire_Crc8(scratchpad, 8U) != scratchpad[8]) return DS18B20_CRC_ERROR;
    /* Nine zero bytes pass CRC: verify fixed bytes and configuration too. */
    if ((scratchpad[4] & 0x9FU) != 0x1FU || scratchpad[5] != 0xFFU ||
        scratchpad[6] != 0x0CU || scratchpad[7] != 0x10U)
        return DS18B20_INVALID_DATA;
    bits = (uint16_t)scratchpad[0] | ((uint16_t)scratchpad[1] << 8U);
    /* Ignore undefined low bits at 9/10/11-bit resolution. */
    bits &= (uint16_t)(0xFFFFU << (3U - ((scratchpad[4] >> 5U) & 3U)));
    raw = (bits & 0x8000U) ? (int32_t)bits - 65536 : (int32_t)bits;
    if (raw < -55 * 16 || raw > 125 * 16) return DS18B20_INVALID_DATA;
    *celsius = (float)raw / 16.0f;
    return DS18B20_OK;
}
```

The first block checks pointers and callbacks. The next starts conversion. The while loop waits with a finite budget. The second reset starts the result read. The for loop collects nine bytes. CRC and fixed-byte checks reject bad data. The last lines assemble the two bytes, handle negative values, check range and write Celsius to the caller.

The 800 nominal millisecond waits are not a wall-clock deadline. Slots, interrupts and HAL tick rounding add overhead; this example can take around 1.7 s on a stuck-busy timeout. It is a

blocking call: the foreground waits, while interrupts can still run between timing-critical portions. Do not call it from an ISR or with interrupts disabled.

2.11 Step 10: understand the adapter when you want to port it

A callback is an ordinary function whose address is given to another function. Instead of including STM32 registers in the portable library, we give it functions meaning "pull low," "release," "read," and "wait." The adapter supplies them.

```
/* Inside the STM32 adapter: */
static void low(void *context)
{
    (void)context;
    ONE_WIRE_1_GPIO_Port->BRR = ONE_WIRE_1_Pin;
}
```

`static` keeps this helper private to its C file. The unused `context` parameter is available for another board to pass a port/pin object; this fixed-pin example does not need it. The GPIO macro comes from CubeMX, which is why pin moves do not require editing this function.

The seven supplied functions have distinct jobs: drive low, release, read high/low, delay in microseconds, save/mask interrupts, restore interrupts, and delay in milliseconds. The bus binding stores their function addresses:

```
void DS18B20_STM32_Init(void)
{
    CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;
    DWT->CYCCNT = 0;
    DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;
    const DS18B20 binding = {
        {NULL, low, release, read_line, delay_us, lock, unlock}, delay_ms
    };
    sensor = binding;
    release(NULL);
}

DS18B20_Status DS18B20_STM32_ReadTemperature(float *celsius)
{
    return DS18B20_ReadTemperature(&sensor, celsius);
}
```

The DWT cycle counter implements short delays: at 72 MHz a microsecond is 72 cycles. HAL

implements millisecond waits. The critical callbacks restore the saved interrupt state instead of blindly enabling interrupts.

The wire layer sends bytes least-significant bit first. Reset requests 480 microseconds low and samples presence 70 microseconds after release. A write-one requests 6 low/64 released; a write-zero 60 low/10 released. A read requests 3 low, 10 after release before sampling, then 53 recovery. GPIO/call overhead must be measured on the actual board. Reset masks interrupts for about 550 microseconds plus overhead, so a system with tighter deadlines needs another timing design.

For an MCU other than this STM32F107, rewrite the adapter and measure timings; keep the portable C files. To extend the API with multiple sensors or asynchronous conversion, first define the behavior and test it. Those features are intentionally outside this first lesson.

2.12 Step 11: record what the board actually proved

The owner reports downloading/running the earlier firmware in Keil and observing temperature at a breakpoint. This is useful initial hardware feedback. It does not yet prove continuous refresh, temperature accuracy, disconnect recovery or measured 1-Wire timing. The corrected live-session setup must be retried on the target.

Record these checks separately:

1. Live refresh on: successes increase without transaction breakpoints.
2. Warm/cool: successive successful readings follow the sensor.
3. Disconnect/reconnect: an error invalidates the result; reconnection recovers.
4. Compare with a reference thermometer and inspect DQ under real interrupt load.
5. Record firmware, supply, crystal, resistor, cable, probe, samples and waveforms.

Put actual observations in PROJECT_STATUS.md. Builds and emulated tests are software evidence; board acceptance needs the observations above. The three C suites use Python Unicorn and PlatformIO's ARM GCC.

3 Add the next independent driver

3.1 Start from a copied foundation

Copy the complete raw template to a new directory named for the IC. Rename its IOC, Keil target and output, then configure hardware in CubeMX. The copied project must own vendor sources and build files. Do not refer to the template with include paths or relative source paths after copying.

Read the official datasheet first and record its document number, revision and official URL in the new README. Keep downloaded PDFs local. Define the smallest useful operation, its output units, timeout behavior and error policy before writing a larger API.

3.2 A repeatable development order

1. Configure pins, clock and power assumptions in the local IOC.
2. Write a small portable header and implementation. Keep register/HAL knowledge in an Application adapter.
3. Add a complete runnable hardware example with visible values and errors.
4. Protect application hooks with USER CODE. Regenerate in a disposable copy and verify preservation.
5. Synchronize Keil, run structural checks, build Keil and PlatformIO and test meaningful protocol/error cases.
6. Write an English-first/Persian standalone README and update local status with actual evidence.
7. Add one chapter in each central report edition, update the repository catalog and rebuild both PDFs.
8. Perform hardware acceptance and append real measurements without replacing the software evidence.

3.3 Add a chapter to this report

Create matching numbered files under both `chapters` directories and add the corresponding `\input` lines before the extension chapter. Keep the contact section at the end. A device chapter follows the same order as DS18B20: purpose, wiring, first run, protocol, API, reasoning, adapter, regeneration, errors, acceptance and reuse. Add official references to both bibliography files and cite them where used.

Use XeLaTeX for both languages, XePersian for Persian and IEEEtran/BibTeX for citation order. The matching color scheme, 1.5 body spacing, small link icons and six author destinations are configured in the edition's `config.tex`. Persian Latin terms are isolated with `\lr` or an LTR environment and placed on separate source lines for easier editing.

From the repository root:

```
python docs/tools/build_reports.py
python docs/tools/verify_reports.py
```

The build runs XeLaTeX, BibTeX and two more XeLaTeX passes. The verifier checks PDF authors, all six author destinations in each page's footer, page bounds and renders pages for review. Inspect the resulting contact sheets and full-size pages as well; automated link checks alone cannot establish good typography. Fonts and render output stay ignored; final PDFs are tracked.

References

[1] Analog Devices / Maxim Integrated, *DS18B20 Programmable Resolution 1-Wire Digital Thermometer*, Jul. 2019, rev. 6. [Online]. Available: https://www.analog.com/media/en/technical-documentation/data-sheets/ds18b20.pdf

[2] Arm / Keil, *uVision User's Guide: Watch Window*, accessed October 12, 2026. [Online]. Available: https://www.keil.com/support/man/docs/uv4cl/uv4cl_db_dbg_watchwin.htm

[3] STMicroelectronics, *STM32CubeMX User Manual and Command-Line Interface*, uM1718, accessed October 12, 2026. [Online]. Available: https://dev.st.com/stm32cube-docs/stm32cubemx/6.18.0/en/docs/markup/CubeMX_CLI.html

About the author and contact

Alireza Khajehvandi

This repository and its teaching guide are maintained under my name. Device specifications and vendor code remain attributed to their original sources.

Portfolio https://alirezaaies.github.io/

GitHub https://github.com/alirezaaies

LinkedIn https://www.linkedin.com/in/alirezakhajehvandi

YouTube https://www.youtube.com/@AlirezaAIES

Telegram https://t.me/AlirezaAIES

Instagram https://www.instagram.com/alireza.aies